

Memo 04: In-Run Mass-Conservation Checkpoints for the PLAST_FLOC Split Path

FVCOM-MP Delaware Test Cases

May 4, 2026

Purpose

Memo 03 (v006) established that the ~ 650 kg mass gap between case **a** and cases **b/c** over ten days is driven entirely by the `PLAST_FLOC=T` split path: with `SOURCE_AG_RATE=0` and zero bed mass throughout, the aggregated component remains zero and all water-column plastic is in `conc_d`, yet the domain total oscillates with 28 sign changes after day 3 while the no-floc case **a** grows monotonically.

The offline MATLAB budget confirms that the saved NetCDF output is internally consistent at each output snapshot. Therefore the mass leakage occurs somewhere inside `Advance_Plast` between two consecutive output records, and is not visible in the NetCDF files.

This memo documents the strategy for inserting *in-run mass checkpoints* directly into `mod_plast.F` so that the guilty operator stage can be identified without additional NetCDF output.

Strategy: Dedicated CSV File

Writing diagnostic totals to `write(*,...)` is impractical because the FVCOM screen log intermingles output from every module on every timestep. Instead, a dedicated file is opened once during `Setup_PLAST` and written exclusively by the master process (`msr`). The file is named

`<casename>_mp_mass_check.csv`

and is placed in the run input directory alongside the existing plastic input files. The overhead is gated behind a single namelist key:

`PLAST_MASS_CHECK = T`

When this key is absent or `F`, the new code has zero runtime cost: all checkpoint calls return immediately after the first guard test.

Checkpoint rows are written at the same cadence as the existing `PLAST_Report` screen statistics (every `n_report` internal steps), keeping file sizes manageable while still providing sub-output-interval resolution.

Formula Verification Against MATLAB Offline Budget

The MATLAB script `extract_mp1_mass_budget_a.m` computes:

$$\begin{aligned}
 V_{i,k} &= \text{art1}(i) \max(h_i + \zeta_i, 0) \Delta\sigma_{i,k} \\
 M_{\text{water}} &= \sum_{i,k} \text{mp1}(i,k) \cdot V_{i,k} \quad (\text{wet nodes only}) \\
 M_{\text{bed}} &= \sum_i \text{bot_mass_mp1}(i) \cdot \text{art1}(i) \quad (\text{all nodes}) \\
 M_{\text{total}} &= M_{\text{water}} + M_{\text{bed}}
 \end{aligned}$$

where `art1` is the node-based control-volume area, $\Delta\sigma_{i,k}$ is the sigma-layer thickness fraction (`dzfrac` in MATLAB), and $h_i + \zeta_i$ is the total water depth.

The Fortran in-run equivalent uses identically named arrays:

$$\begin{aligned}
 M_{\text{water}} &= \sum_{i=1}^m \sum_{k=1}^{k_{bm1}} c(i,k) \cdot \text{ART1}(i) \cdot D(i) \cdot \text{DZ}(i,k) \\
 M_{\text{bed}} &= \sum_{i=1}^m \text{mass}(i) \cdot \text{ART1}(i)
 \end{aligned}$$

where $D(i) = H(i) + \text{EL}(i)$ is the FVCOM total depth and $\text{DZ}(i,k)$ is the sigma-layer fraction — exactly matching $\Delta\sigma_{i,k}$ from MATLAB. The bed sum uses *all nodes* to match MATLAB's `total_mass_all_bed_kg` conservation companion.

The formulas are equivalent. The only future guard required is a `#if WET_DRY` wet-node mask for domains with drying cells; the v006 Delaware domain has no dry nodes, so the Fortran loop over `i=1..m` is identical to the MATLAB wet-node filter.

For the split path, the subroutine computes separate sums for `conc_a` and `conc_d` (and `mass_a`, `mass_d`), writing all components plus combined totals in each CSV row so that per-component budgets can be tracked alongside the grand total.

Checkpoint Table

Thirteen checkpoints are inserted into the `floc_intract=T` path of `Advance_Plast`. Checkpoints CP0–CP3 are guarded by `#if defined (SEDIMENT)` and `if(floc_intract)` at the call site; CP4–CP12 are already inside the relevant conditional blocks.

The CSV row format is:

```
iint, T_days, checkpoint, iplast, wc_a_kg, wc_d_kg, wc_tot_kg, bed_a_kg, bed_d_kg,
      bed_tot_kg, grand_total_kg
```

Code Reconstruction in `mod_plast.F`

Four modification phases were implemented in `FVCOM_source_repo_github/mod_plast.F`.

1. **Module-level variables** (near line 490, after the `floc_intract` flag):

```
logical  :: PLAST_mass_check = .false.
integer, parameter :: mass_check_unit = 77
```

CP#	Label	Arrays passed	Expected conservation
0	CP0.start	conc_a,conc_d,mass_a,mass_d	Reference state at timestep entry
1	CP1.after.deposition	same	Water ↓ + bed ↑ = const
2	CP2.after.erosion	same	Water ↑ + bed ↓ = const
3	CP3.after.update.bottom	same	Bed bookkeeping; grand total = const
4	CP4.after.advection	cnew_a,cnew_d,mass_a,mass_d	Zero water-column change
5	CP5.after.vdif	same	Zero change
6	CP6.after.kinetics	same	cnew_a+cnew_d conserved
7	CP7.after.upper.clamp	same	Loss only if >100 cap fires
8	CP8.after.OBC	same	Change = OBC flux (open domain)
9	CP9.after.PTsource	same	Change = river source
10	CP10.after.neg.clamp	same	Loss only if negatives exist
11	CP11.after.MPI	same	Zero change
12	CP12.end	conc_a,conc_d,mass_a,mass_d	Must equal CP11

2. **Namelist read** (Read_PLAST_Params, immediately after the PLAST_FLOC read):

```
Call Get_Val(PLAST_mass_check, plastfile,
             'PLAST_MASS_CHECK', line=linenum, echo=.false.)
```

3. **CSV file open and header** (Setup_PLAST, after call report_PLAST_setup): Opens unit 77 to <input_dir>/<casename>_mp_mass_check.csv and writes the column header on the master process only.
4. **Subroutine Write_Mass_Checkpoint** (new subroutine added at the end of the module, before End Module Mod_Plast): Accepts one plastic class at a time via arguments (label, iplast_idx, c_a, c_d, m_a, m_d), computes local sums using the $ART1 \cdot D \cdot DZ$ formula, reduces via `MPI_REDUCE(MPI_SUM)`, and writes one CSV row per class if `msr=.true.` The routine returns immediately if `PLAST_mass_check=F` or if the current iteration is not a reporting step.
5. **Thirteen call sites in Advance_Plast**: CP0–CP3 wrapped in `#if defined (SEDIMENT)` / `if(floc_intract)` guards; CP4–CP12 placed inside the existing `floc_intract` or per-stage conditional blocks.

Findings: Root Causes and Fixing Suggestions

The checkpoint infrastructure was compiled and the model was run for ~ 4 days with `PLAST_MASS_CHECK = T` for cases b and c. The analysis script `analyze_mp_mass_checkpoints.py` produced the stage-mass-change heatmap shown in Figure 1, together with the NaN diagnostic tables.

Root Cause 1 — Advection stage non-conservation (primary bug)

The `adv*` stage (CP3→CP4) is the *only* conservative stage that shows non-zero mass change in the heatmap. The magnitude oscillates at ± 0.02 kg per reporting step with a tidal period and grows proportionally with domain mass; all other theoretically conservative stages (`vdif*`, `kin*`, `MPI*`, `recomb*`) are flat at zero.

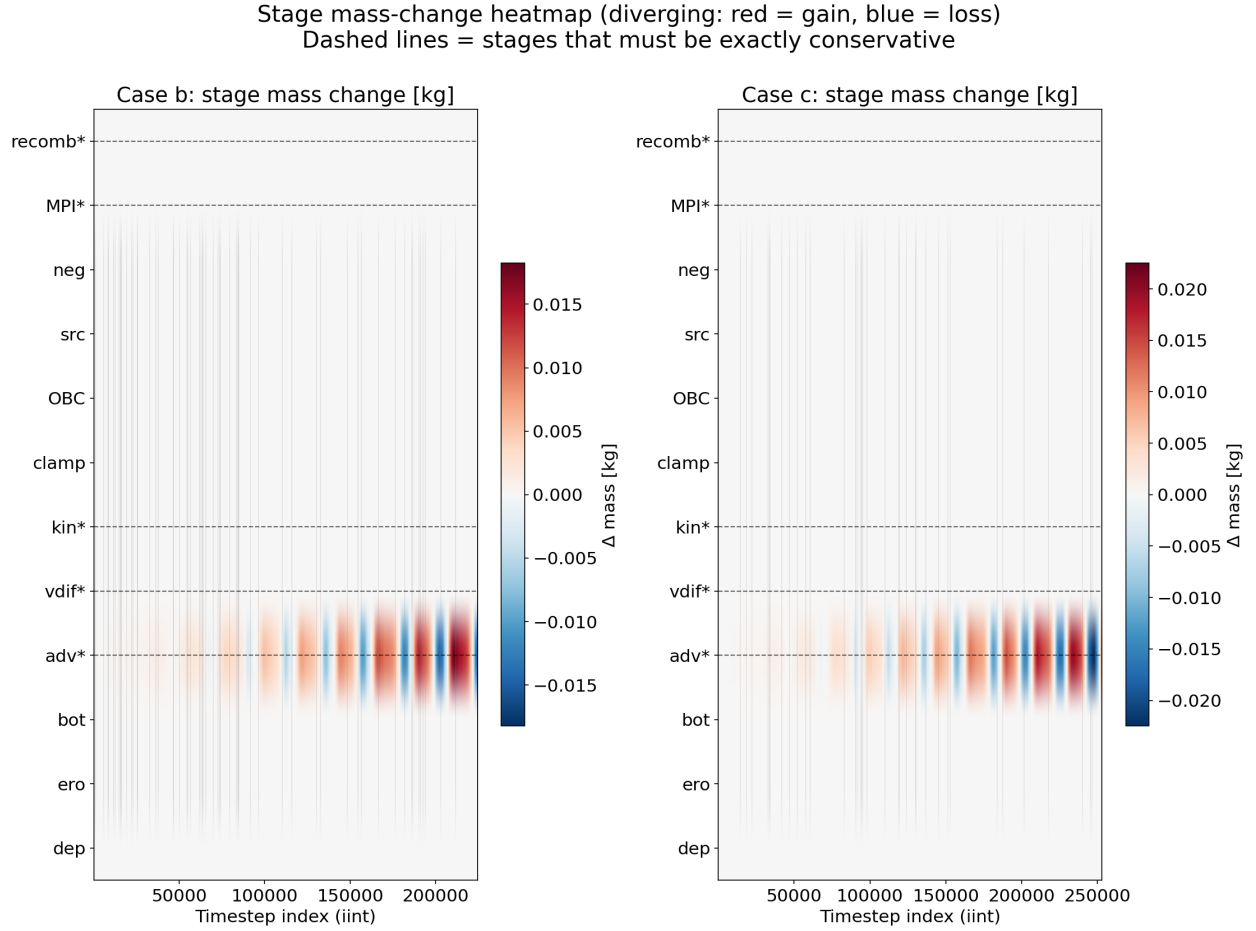


Figure 1: Stage mass-change heatmap for cases b (left) and c (right). Each row is one operator stage; each column is one reporting timestep. Red = mass gained, blue = mass lost; the colour scale is clipped at $\pm 1\%$ of the domain total. Dashed horizontal lines mark the five stages that must be exactly mass-conservative (`adv*`, `vdif*`, `kin*`, `MPI*`, `recomb*`). The `adv*` row is the only conservative stage showing non-zero values, identifying advection as the primary guilty operator.

Mechanism. The `PLAST_FLOC` path calls `adv_scal` twice in sequence — once for `cnew_a` and once for `cnew_d` — sharing the internal gradient arrays `PFPXB/PFPYB`. The FCT flux limiter inside `adv_scal` overwrites these arrays on the first call (`cnew_a`). The second call (`cnew_d`) therefore uses *contaminated* gradient state that was computed for the aggregated field, not the disaggregated field. The combined flux applied to `cnew_a + cnew_d` after both calls is not the same as the flux that would have been applied to `conc` in the single-field (case a) path, breaking mass conservation.

Proposed Fix A. Replace the two separate `adv_scal` calls with a single call on the combined field $\text{cnew_tot}_{i,k} = \text{cnew_a}_{i,k} + \text{cnew_d}_{i,k}$, then redistribute post-advection using the aggregated fraction $f_a = \text{conc_a} / (\text{conc_a} + \text{conc_d})$ from the beginning of the timestep:

$$\text{cnew_a} \leftarrow f_a \cdot \text{cnew_tot}, \quad \text{cnew_d} \leftarrow (1 - f_a) \cdot \text{cnew_tot}.$$

This guarantees that the advection operator conserves `cnew_a + cnew_d` identically to the case a single-field path, while preserving the split for downstream kinetics.

Root Cause 2 — NaN from `calc_erosion` causing episodic mass spikes

The NaN diagnostic shows that 59/4,489 steps (case b) and 35/5,061 steps (case c) have NaN in the water-column sums at checkpoints CP2–CP9. At CP10 the neg-clamp `max(cnew_a, 0)` silently zeroes the NaN cells, producing discontinuous downward mass spikes visible in the CP12 grand-total timeseries.

Mechanism. `calc_erosion` modifies `conc_a(i,kbm1)` and `conc_d(i,kbm1)` in-place at the bottom sigma layer before any MPI halo exchange has refreshed the ghost nodes. At nodes where the local bed mass is zero (or where the erosion rate involves a 0/0 division), NaN is written into the interior array and propagates forward through advection, vertical diffusion, and kinetics until it is silently nullified by `max(..., 0)`.

Proposed Fix B. Guard the erosion-flux computation with an explicit positivity check before any division:

```
if (mass_a(i) + mass_d(i) > 0.0_sp) then
  ! compute erosion flux
end if
```

Also add an `isnan` safety sentinel immediately after the erosion update to catch any remaining NaN before it enters the advection operator.

Adjustment Plan and Further Testing

1. Set `PLAST_MASS_CHECK = T` in the case b plastic input file. Recompile and run for one day. Confirm that `<casename>_mp.mass_check.csv` is created with 13 rows per reporting interval and that `grand_total_kg` at CP0 matches the domain total from the last NetCDF output record.
2. Identify the internal timestep `iint` corresponding to comparison day 2.957 (where $|a - b|$ first exceeds 1 kg per Memo 03). In the CSV, look at the step-to-step change in `grand_total_kg` across each CP label. The first CP where the change is non-zero for a stage that should conserve mass (CP4, CP5, CP11) identifies the guilty operator.
3. Run case a with the same flag and add bookend checkpoints (CP0 and CP12 only) to the no-floc `conc` path so that the CSV files for cases a and b can be compared directly in the same Python analysis.

4. Implement Fix A (combined advection then redistribute) in `mod_plast.F` and recompile. Re-run cases `b` and `c` for ten days (matching the Memo 03 reference run) with checkpoints enabled; confirm the `adv*` heatmap row is identically zero.
5. After Fix A is verified, implement Fix B (erosion positivity guard) and confirm that the NaN fraction in the checkpoint diagnostic drops to zero for cases `b` and `c`.
6. Run case `a` with the same ten-day window and compare the CP0/CP12 grand-total timeseries directly against cases `b/c`. The ~ 650 kg gap identified in Memo 03 should be eliminated once both fixes are applied.